



CAPSTONE PROJECT | AEROLOGISTICS

Project Overheat

Predictive Maintenance AI

Predicting drive-motor failures 24 hours before they happen

Data Science Internship · Time-Series Machine Learning

The Problem: Reactive Maintenance

\$50,000

lost every minute a fulfillment center
is shut down

in delayed shipments



What is happening

Drive motors in high-speed sorting robots overheat and fail without warning, halting whole warehouses.



Today: reactive

We repair after the breakdown. Every failure is a catastrophic, unplanned shutdown.



Goal: proactive

Use IoT telemetry to predict a failure at least 24 hours ahead so mechanics can fix it on their schedule.

Data & Four Production Traps

10,000 hourly logs, but only 15 failure hours (0.15%): a severe class imbalance



1. The Windowing Trap

One jolt is noise; sustained shaking is degradation. Use rolling means and standard deviations.



2. The Lead-Time Trap

Predicting the failure moment is useless. Shift the target so the model predicts 24 hours ahead.



3. The Data Leakage Trap

Random shuffling lets the model learn from the future. Split strictly by time.

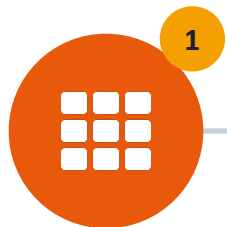


4. Explainable AI

Mechanics won't replace a \$10,000 motor blindly. Prove which sensor leads.

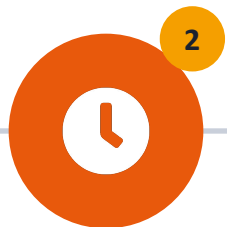
Solution Pipeline

Four phases, run linearly in one notebook



Feature Engineering

12-hour rolling windows



Target Shifting

Label = failure in 24 h



Chronological Split

First 80% train, last 20% test



Model & Explain

Balanced Random Forest +
importances

Phase 1: Rolling-Window Features

Capture sustained degradation, not one-off spikes



Sort chronologically

Parse the timestamp and order rows in time.



12-hour rolling mean of temperature

Reveals sustained heating (required feature).



12-hour rolling std of vibration

Reveals erratic shaking (required feature).



Drop the first NaN rows

Windows are backward-looking, so no future data is used.

```
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"])
df = df.sort_values("Timestamp")

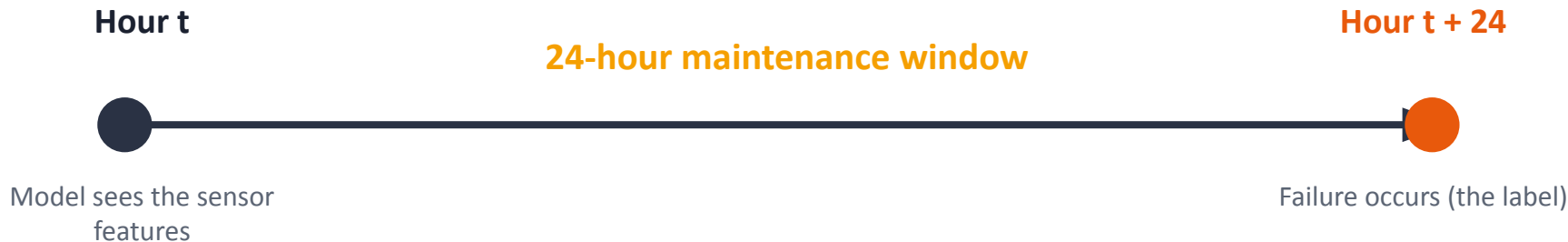
for short, col in sensors.items():
    roll = df[col].rolling(12)
    df[f"{short}_Rolling_Mean_12h"] = roll.mean()
    df[f"{short}_Rolling_Std_12h"] = roll.std()

df = df.dropna()    # first 11 rows
```

Extended to mean and std for all three sensors, so temperature, vibration and voltage compete fairly in the explainability phase.

Phase 2: The 24-Hour Lead Time

Pair today's sensor readings with tomorrow's outcome



Why shift?

Without the shift the model just says "the machine is breaking right now", which no mechanic can use.

```
df["Failure_In_24_Hours"] = \
    df["Catastrophic_Failure"].shift(-24)
df = df.iloc[:-24] # null targets
df = df.drop(columns=[
    "Catastrophic_Failure"]) # no leakage
```

Phase 3: Chronological Split

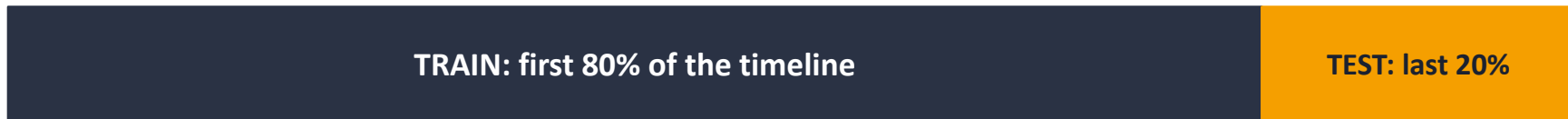
Never let the model see the future

Random shuffle (wrong)



Train and test rows are mixed, so the model learns from the future to predict the past.

Chronological slice (used)



9,965 usable rows: train 7,972 (Jan–Nov 2023, 12 failures) | test 1,993 (Nov 2023–Feb 2024, 3 failures)

```
split_idx = int(len(df) * 0.80) → X.iloc[:split_idx] / X.iloc[split_idx:]
```

Phase 4: Modeling for Recall

Balanced Random Forest with a recall-oriented threshold



Model setup

- RandomForestClassifier, 300 trees
- `class_weight = "balanced_subsample"` penalizes missed failures
- Threshold from time-series cross-validation on training data only: highest value with 75% out-of-fold recall (0.10)
- Test set is never used for tuning

Why recall matters most

Missed failure

\$50,000 / min

False alarm

One extra inspection

So we accept extra false alarms to catch more real failures.

Results on the Held-Out Test Period

Final 20% of the timeline (Nov 2023 to Feb 2024), never seen in training

2 of 3

failures caught 24 h ahead

Recall 67%

10

alerts raised in 1,993 hours

8 false alarms

20%

of alerts were real failures

Precision

Confusion matrix

	Predicted normal	Predicted failure
Actual normal	1,982	8
Actual failure	1	2

How to read this

Only 3 failures fall in the test window, so this is a small-sample estimate. About one false alarm per 250 hours is a cheap price against \$50,000 per minute of downtime.

Explainable AI: Vibration Leads

Random Forest feature importances (mean decrease in impurity)

Importance by sensor

 Vibration **63.4%**

 Temperature **31.5%**

 Voltage **5.0%**

Voltage is a weak indicator; vibration and temperature drive the warning.

Recommendation for the Maintenance Crew



Watch vibration first

It carries 63% of the predictive power, ahead of temperature (32%) and voltage (5%).



Alert on sustained trends

Use 12-hour windows, not single spikes, to avoid alarm fatigue.



Act inside the 24-hour window

Schedule the bearing and cooling check at the next shift change.



Keep improving the model

Only 15 failures exist in the data. Retrain monthly as more history arrives.

From reactive repairs to proactive maintenance.